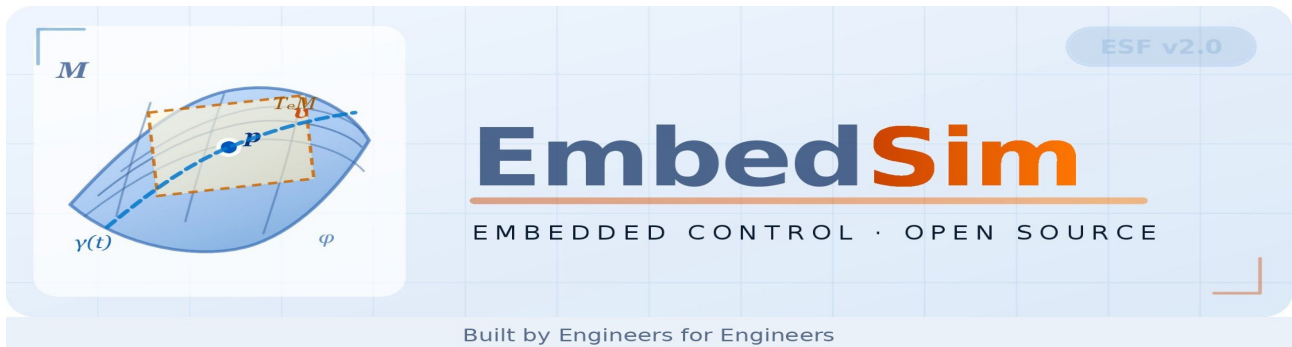

EmbedSim CodeGen

Technical Reference — Feature 05121967

PYXInspector · LoopGenerator · StepGenerator



NANOTEC DB42S02 — Open-loop V/f PMSM drive
AURIX TC3xx target · MISRA C:2012 · ISO 26262 ASIL-D
Revision 1.0 — March 2026

1	Introduction.....	4
2	Engineering Sequence — Python Comes First, Always.....	4
3	PYXInspector.....	4
3.1	Purpose.....	4
3.2	Six CodeGen Attributes.....	4
3.3	How auto_populate_from_pyx Works.....	4
3.4	C_INPUT_MAP — Explicit Port Wiring.....	5
4	LoopGenerator.....	5
4.1	Purpose.....	5
4.2	Calling generate_loop().....	5
4.3	Generated embedsim_loop.c — Annotated Excerpt.....	5
4.4	Zero Python-only Comments Rule.....	6
5	StepGenerator.....	6
5.1	Purpose.....	6
5.2	Calling generate_step().....	6
5.3	Generated Header — EmbedSim_step.h.....	6
6	AURIX GTM ATOM Integration.....	7
6.1	Channel Assignment — DB42S02.....	7
6.2	Why ATOM, Not TOM.....	7
6.3	Bare-Metal C — ATOM Duty Write.....	7
7	FOC α - β Phasor Visualisation.....	7
7.1	Phase-Axis Projections — Clarke Transform.....	7
7.2	SVPWM Sector Convention.....	7
8	Motor Parameters — NANOTEC DB42S02.....	8
9	Generated File Map.....	8
10	Glossary.....	8

1 Introduction

EmbedSim is an open-source Python-first simulation and code-generation framework targeting AURIX TriCore and ARM Cortex-M platforms. Feature 05121967 introduces an automated C code-generation pipeline that transforms a Python signal-flow graph into production-ready, MISRA C:2012-compliant C source files with zero hand-written C strings in the simulation file.

The pipeline has three components: PYXInspector, LoopGenerator, and StepGenerator. Together they emit `embedsim_loop.c / .h` and `EmbedSim_step.c / .h` into the `embedsim_gen/` folder.

2 Engineering Sequence — Python Comes First, Always

Golden Rule

1. Design and verify the complete algorithm in Python (VectorBlock chain).
2. Simulate until steady-state behaviour is confirmed.
3. Only then invoke code generation to emit C.

Never write C first and retrofit Python. The Python model IS the specification.

The canonical block-diagram order for the DB42S02 open-loop V/f controller is:

```
CodeGenStart
|
SpeedRampBlock      → omega_m_ref [rad/s]
VfAngleBlock        → [v_d, v_q, theta_e]
VfDQBlock           → [v_d, v_q]
VfThetaBlock        → theta_e
InvParkTransformBlock → [v_alpha, v_beta]
SVPWMBlock          → [T1, T2, T0, sector]
DutyPackBlock       → [duty_a, duty_b, duty_c, v_dc, 0]
CodeGenEnd           → cg_end.generate_loop() / generate_step()
```

3 PYXInspector

3.1 Purpose

PYXInspector auto-populates six CodeGen attributes on every VectorBlock subclass at class-definition time, using Python's `__init_subclass__` hook. No hand-written C strings are required in the simulation file.

3.2 Six CodeGen Attributes

Attribute	C Type	Description
<code>C_STEP_FUNC</code>	<code>const char*</code>	Name of the C step function, e.g. <code>InvPark_Step</code>
<code>C_STATE_STRUCT</code>	<code>const char*</code>	Name of the persistent state struct type
<code>C_INIT_FUNC</code>	<code>const char*</code>	Name of the C initialisation function
<code>C_INPUT_MAP</code>	<code>dict</code>	Port → signal-name mapping for explicit wiring
<code>C_OUTPUT_NAMES</code>	<code>list[str]</code>	Output signal names in port order
<code>C_CUSTOM_EMIT</code>	<code>str None</code>	Escape hatch: custom C fragment (use sparingly)

3.3 How `auto_populate_from_pyx` Works

When a VectorBlock subclass is defined with a `PYX_FILE` class attribute, `__init_subclass__` calls `PYXInspector.auto_populate_from_pyx(cls)`. The inspector parses the `.pyx` file and fills the six

attributes from the cdef / def signatures found there.

```
# electrical_blocks/pyx_inspector.py
class PYXInspector:
    @staticmethod
    def auto_populate_from_pyx(cls: type) -> None:
        pyx = Path(cls.PYX_FILE)
        step = _find_step_func(pyx)      # e.g. 'InvPark_Step'
        state = _find_state_struct(pyx)  # e.g. 'InvPark_State_T'
        init = _find_init_func(pyx)      # e.g. 'InvPark_Init'
        cls.C_STEP_FUNC = step
        cls.C_STATE_STRUCT = state
        cls.C_INIT_FUNC = init
        # C INPUT_MAP and C OUTPUT_NAMES set per-block in subclass body
```

3.4 C_INPUT_MAP — Explicit Port Wiring

Multi-port blocks require an explicit map so LoopGenerator can wire each input signal by name rather than by positional index. The map is written once in the subclass definition.

```
class InvParkTransformBlock(VectorBlock):
    PYX_FILE = 'coordinate_transform_blocks.pyx'
    C_INPUT_MAP = {
        0: 'vdq',      # port 0 → InvPark input: [v_d, v_q]
        1: 'theta_e',  # port 1 → electrical angle
    }
    C_OUTPUT_NAMES = ['v_alpha', 'v_beta']
```

4 LoopGenerator

4.1 Purpose

LoopGenerator traverses the topologically sorted block list between CodeGenStart and CodeGenEnd, and emits a complete C loop body: embedsim_loop.c and embedsim_loop.h.

Every block contributes exactly one typed C call. Zero Python-only comments appear in the generated source. Zero C_CUSTOM_EMIT entries are required for standard blocks.

4.2 Calling generate_loop()

```
cg_end.generate_loop(
    cg_start = cg_start,
    output_dir = _ROOT,
    dt_hz = 1.0 / DT,      # e.g. 10000 Hz for DT=1e-4
    write_files = True,
)
```

4.3 Generated embedsim_loop.c — Annotated Excerpt

```
/* embedsim_loop.c — generated by EmbedSim LoopGenerator */
/* DO NOT EDIT — regenerate from Python simulation */

#include "embedsim_loop.h"
#include "motor_utility_blocks.h"
#include "coordinate_transform.h"
#include "svpwm.h"

static SpeedRamp_State_T speed_ref_state;
static VfAngle_State_T vf_angle_state;
static InvPark_State_T inv_park_state;
static SVPWM_State_T svpwm_state;
static DutyPack_State_T duty_pack_state;
```

```

void EmbedSim_Loop_Init(void) {
    SpeedRamp_Init(&speed_ref_state);
    VfAngle_Init(&vf_angle_state);
    InvPark_Init(&inv_park_state);
    SVPWM_Init(&svpwm_state);
    DutyPack_Init(&duty_pack_state);
}

void EmbedSim_Loop_Step(float dt) {
    SpeedRamp_Step (&speed_ref_state, dt, &omega_ref);
    VfAngle_Step   (&vf_angle_state, dt, omega_ref, &vdq, &theta_e);
    InvPark_Step    (&inv_park_state, vdq, theta_e, &v_alpha, &v_beta);
    SVPWM_Step      (&svpwm_state,    v_alpha, v_beta, V_DC,
                    &T1, &T2, &T0, &sector);
    DutyPack_Step   (&duty_pack_state, v_alpha, v_beta,
                    T1, T2, T0, sector, &duty_a, &duty_b, &duty_c);
}

```

4.4 Zero Python-only Comments Rule

MISRA Compliance Note The generated C files must contain zero Python-model comments. All comments are engineering annotations written in C block style. Rationale: Python docstrings and EmbedSim framework comments are not part of the deliverable and would confuse static-analysis tools.

5 StepGenerator

5.1 Purpose

StepGenerator emits a Simulink-equivalent step function wrapper: EmbedSim_step.c / .h. It reads the signals at CodeGenStart for the input struct and the signals at CodeGenEnd for the output struct.

5.2 Calling generate_step()

```

cg_end.generate_step(
    cg_start      = cg_start,
    output_dir     = _ROOT,
    dt_hz          = 1.0 / DT,
    prefix         = 'EmbedSim',    # → EmbedSim_Input_T / EmbedSim_Output_T
    write_files    = True,
)

```

5.3 Generated Header — EmbedSim_step.h

```

/* EmbedSim_step.h */
#pragma once
#include <stdint.h>

typedef struct {
    float omega_ref_rads;    /* rad/s — from SpeedRampBlock port 0 */
} EmbedSim_Input_T;

typedef struct {
    float duty_a;            /* 0..1 — ATOM0 CH0 */
    float duty_b;            /* 0..1 — ATOM0 CH2 */
    float duty_c;            /* 0..1 — ATOM0 CH4 */
} EmbedSim_Output_T;

void EmbedSim_Step(float dt,
                  const EmbedSim_Input_T *in,

```

```
EmbedSim_Output_T *out);
```

6 AURIX GTM ATOM Integration

6.1 Channel Assignment — DB42S02

Signal	ATOM	Channel	Note
duty_a	ATOM0	CH0	Phase A — upper + lower
duty_b	ATOM0	CH2	Phase B — upper + lower
duty_c	ATOM0	CH4	Phase C — upper + lower

6.2 Why ATOM, Not TOM

TOM (Timer Output Module) is a 16-bit fixed-resolution module suited for simple GPIO toggling. ATOM (Advanced Timer Output Module) provides 24-bit resolution, CMU clock input, and full ARU (Action Request Unit) interconnect — mandatory for motor-control PWM with sub-cycle dead-time insertion on AURIX TC3xx.

6.3 Bare-Metal C — ATOM Duty Write

```
/* Write EmbedSim output to GTM ATOM channels */
void Motor_WriteDuty(const EmbedSim_Output_T *out) {
    uint32 period = GTM_ATOM0_CH0_SR0; /* shadow register = period */
    GTM_ATOM0_CH0_SR1 = (uint32)(out->duty_a * (float)period);
    GTM_ATOM0_CH2_SR1 = (uint32)(out->duty_b * (float)period);
    GTM_ATOM0_CH4_SR1 = (uint32)(out->duty_c * (float)period);
}
```

7 FOC α - β Phasor Visualisation

The animation (db42s02_phasor_anim.gif) illustrates the connection between three-phase currents and the rotating voltage phasor in α - β space.

7.1 Phase-Axis Projections — Clarke Transform

Each phase current is projected onto its fixed spatial axis in α - β space:

```
phase a → axis at 0 deg (along +alpha)
phase b → axis at -120 deg
phase c → axis at +120 deg

Spoke length = (i_x / i_peak) * R_display (signed)

The vector sum of the three signed spokes == the voltage phasor direction.
This is the geometric core of space-vector theory.
```

7.2 SVPWM Sector Convention

Standard IEC / Texas Instruments / Infineon convention: Sector 1 is centred on the $+\alpha$ axis (0°), with boundaries at $\pm 30^\circ$. NOT at 0° , 60° , 120° (a common off-by-one error).

Sector	Centre angle	Boundary	Dominant phases
S1	0°	$-30^\circ \dots +30^\circ$	$+i_a$, returning i_b and i_c
S2	60°	$+30^\circ \dots +90^\circ$	$+i_a$ and $+i_b$, returning i_c
S3	120°	$+90^\circ \dots +150^\circ$	$+i_b$, returning i_a and i_c
S4	180°	$+150^\circ \dots +210^\circ$	$-i_a$, returning i_b and i_c
S5	240°	$+210^\circ \dots +270^\circ$	$-i_a$ and $-i_b$, returning i_c

Sector	Centre angle	Boundary	Dominant phases
S6	300°	+270° ... +330°	-i_b, returning i_a and i_c

8 Motor Parameters — NANOTEC DB42S02

Parameter	Value	Description
Stator resistance R	0.19 Ω	Phase resistance
d/q inductance $L_d = L_q$	0.125 mH	Surface PMSM ($L_d = L_q$)
PM flux linkage λ_{pm}	0.0014 Wb	Permanent magnet flux
Inertia J	$2.4 \times 10^{-6} \text{ kg} \cdot \text{m}^2$	Rotor moment of inertia
Pole pairs p	4	Electrical / mechanical ratio
DC bus V_dc	17 V	Inverter supply voltage
Rated speed	8000 RPM	Open-loop V/f reference
V/f ratio	$V_{\text{phase_peak}} / \omega_{e_rated}$	Scalar control law

9 Generated File Map

File	Contents
embedsim_gen/embedsim_loop.c	Full controller loop: state vars, Init(), Step()
embedsim_gen/embedsim_loop.h	Public API: EmbedSim_Loop_Init / _Step
embedsim_gen/EmbedSim_step.c	Simulink-equivalent wrapper calling Loop_Step
embedsim_gen/EmbedSim_step.h	EmbedSim_Input_T, EmbedSim_Output_T, Step() proto
db42s02_topology.html	Interactive signal-flow graph (TopologyPrinter)
db42s02_phasor_anim.gif	Animated phasor + phase currents + PWM bars

10 Glossary

Term	Definition
ATOM	Advanced Timer Output Module — GTM sub-module, 24-bit, CMU input, ARU-connected
ARU	Action Request Unit — GTM interconnect fabric for zero-CPU signal routing
CMU	Clock Management Unit — GTM clock source for ATOM/TOM channels
CodeGen	EmbedSim code-generation pipeline (feature 05121967)
DXA	Document eXact Ansi — 1/20 of a point; 1440 DXA = 1 inch
FMU	Functional Mock-up Unit — FMI 2.0 plant model from Modelica
FOC	Field-Oriented Control — dq-frame current regulation
GTM	Generic Timer Module — AURIX TC3xx PWM and timer subsystem
InvPark	Inverse Park transform: dq $\rightarrow$ $\alpha\beta$ space
ISO 26262	Functional Safety standard for road vehicles
LoopGen	LoopGenerator — emits embedsim_loop.c / .h
MISRA C	Motor Industry Software Reliability Association C coding standard
PMSM	Permanent Magnet Synchronous Motor
PYXInspector	EmbedSim class that parses .pyx files to fill CodeGen attributes
StepGen	StepGenerator — emits EmbedSim_step.c / .h
SVPWM	Space Vector PWM — $\alpha\beta$ voltage modulation algorithm
TOM	Timer Output Module — GTM sub-module, 16-bit, no ARU (not used for motor control)
V/f control	Scalar open-loop speed control: voltage proportional to frequency
VectorBlock	EmbedSim base class for all signal-processing blocks